

```

import numpy as np

# Define the input vectors
vectors = np.array([[1, -1, -1, -1],
                    [-1, 1, -1, -1],
                    [-1, -1, 1, 1],
                    [-1, -1, 1, 1]])

# Define the weight matrix
weights = np.zeros((4, 4))
for i in range(4):
    for j in range(4):
        if i == j:
            weights[i][j] = 0
        else:
            weights[i][j] = (vectors[i] @ vectors[j]) / 4

# Define the activation function
def activation_function(x):
    if x >= 0:
        return 1
    else:
        return -1

# Define the Hopfield Network function
def hopfield_network(x, weights):
    y = np.copy(x)
    for i in range(4):
        sum_val = 0
        for j in range(4):
            sum_val += weights[i][j] * y[j]
        y[i] = activation_function(sum_val)
    return y

# Test the Hopfield Network function
for i in range(4):
    print("Input vector:", vectors[i])
    output = hopfield_network(vectors[i], weights)
    print("Output vector:", output)
    print()

```

↩ Input vector: [1 -1 -1 -1]
Output vector: [1 1 -1 -1]

Input vector: [-1 1 -1 -1]
Output vector: [1 1 -1 -1]

Input vector: [-1 -1 1 1]
Output vector: [-1 -1 1 1]

Input vector: [-1 -1 1 1]
Output vector: [-1 -1 1 1]